BTK AKADEMİ

İLERİ SEVİYE PYTHON PROGRAMLAMA DİLİ

Nesnelerin Hiyerarşisi ve Mimarisi

EBRU AYDOĞAN

Inheritance (Kalıtım)

- **Problem:** Bir yazılımda Ogrenci ve Ogretmen sınıflarınız olduğunu hayal edin. İkisinin de ad, soyad, yas bilgileri var. Her iki sınıf için de ayrı ayrı __init__ yazmak, aynı kodları tekrar tekrar yazmak (spagetti kod) demektir.
- **Kalıtım (Inheritance):** Bir sınıftaki ortak özellikleri, yeni sınıflara "miras" olarak aktarma yeteneğidir.
- **Avantajı:** DRY (Don't Repeat Yourself - Kendini Tekrar Etme) prensibine uyum sağlarız. Kodumuz daha bakımlı ve modüler olur.

Inheritance (Kalıtım)

- Temel Sınıf (Base Class)
  - Önce hiyerarşinin en tepesindeki "Genel" sınıfı tanımlıyoruz. Bu sınıf, herkesin ortak özelliklerini tutan "ata" sınıfımızdır.

```
class Person:
    def __init__(self, name, surname, age):
        self.name = name
        self.surname = surname
        self.age = age
        print("Person nesnesi üretildi.")

    def who_am_i(self):
        print("Ben bir insanım.")
```

BTK AKADEMİ

Inheritance (Kalıtım)

- Çocuk Sınıflar (Child Classes)
  - Şimdi Person sınıfından türeyen özelleşmiş sınıfları yazıyoruz.

```
class Student(Person):
    def __init__(self, name, surname, age, student_number):
        # 'super()' ile Person'ın init'ini tetikliyoruz (miras aldık)
        super().__init__(name, surname, age)
        self.student_number = student_number
        print("Student nesnesi üretildi.")

class Teacher(Person):
    def __init__(self, name, surname, age, branch):
        super().__init__(name, surname, age)
        self.branch = branch
```

BTK AKA

Metot Ezme (Overriding)

- Üst sınıftan gelen bir metodu, çocuk sınıfın kendi ihtiyaçlarına göre değiştirmesidir.
- **super()**: "Benim babamın (üst sınıfın) şu metodunu veya özelliğini kullan, ama üzerine şunları da ekle" demenin en profesyonel yoludur.

```
# Student sınıfına kendi who_am_i'sini yazalım (Overriding)
class Student(Person):
    # ... init kısmı yukarıdaki gibi ...

    def who_am_i(self):
        print("Ben bir öğrenciyim.")

s1 = Student("Ali", "Yılmaz", 20, 101)
s1.who_am_i() # "Ben bir insanım" yerine "Ben bir öğrenciyim" yazar.
```

BTK AKADEMİ

Polymorphism (Çok Biçimlilik)

- **Mantığı:** Farklı sınıflardan türetilmiş nesnelerin, aynı isimli bir metodu çağırıldığında **kendi doğalarına göre** farklı tepkiler vermesidir.
- **Analoji:** Bir "Spor Yap" butonu düşünün.
  - Bir *Yüzücü* nesnesine "Spor Yap" derseniz: Havuza atlar.
  - Bir *Koşucu* nesnesine "Spor Yap" derseniz: Ayakkabılarını bağlayıp koşmaya başlar.
  - Komut aynı ("Spor Yap"), ama sonuç nesneye göre değişiyor.

Polymorphism (Çok Biçimlilik)

- Polymorphism Olmadan (Eski Yöntem)
- Polymorphism'i bilmeyen bir yazılımcı, her nesneyi tek tek kontrol etmek zorunda kalır. Bu da kod kirliliğine yol açar:

```
# Kötü Uygulama: Her nesneyi tek tek kontrol etmek (if-else karmaşası)
hayvanlar = [kedi, kopek, kus]

for h in hayvanlar:
    if isinstance(h, Kedi):
        print("Miyav")
    elif isinstance(h, Kopek):
        print("Hav")
    elif isinstance(h, Kus):
        print("Cik")
```

BTK AKADEMİ

Polymorphism (Çok Biçimlilik)

- Polymorphism ile (Profesyonel Yöntem)
- Şimdi tüm hayvanlara aynı isimde (speak) bir metot veriyoruz ve Python'ın bu metotları nesneye göre ayırt etmesini sağlıyoruz.

```
class Animal:
    def speak(self):
        raise NotImplementedError("Subclass must implement this method")

class Dog(Animal):
    def speak(self):
        return "Hav Hav!"

class Cat(Animal):
    def speak(self):
        return "Miyav!"
```

BTK AKADEMİ

Polymorphism (Çok Biçimlilik)

- Polymorphism ile (Profesyonel Yöntem)
- Şimdi tüm hayvanlara aynı isimde (speak) bir metot veriyoruz ve Python'ın bu metotları nesneye göre ayırt etmesini sağlıyoruz.

```
# Artık tek bir döngüde hepsini çalıştırabiliriz:
hayvanlar = [Dog(), Cat(), Dog()]

for h in hayvanlar:
    print(h.speak()) # İf-else yok, kontrol yok! Nesne kendi sesini biliyor.
```

BTK AKADEMİ

Polymorphism (Çok Biçimlilik)

- Polymorphism ile (Profesyonel Yöntem)
- Şimdi tüm hayvanlara aynı isimde (speak) bir metot veriyoruz ve Python'ın bu metotları nesneye göre ayırt etmesini sağlıyoruz.

```
# Artık tek bir döngüde hepsini çalıştırabiliriz:
hayvanlar = [Dog(), Cat(), Dog()]

for h in hayvanlar:
    print(h.speak()) # İf-else yok, kontrol yok! Nesne kendi sesini biliyor.
```

BTK AKADEMİ

Meta Class Kavramı

- Python'da Sınıflar Nasıl Oluşur?
- **Soru:** Biz class Person: yazdığımızda Python arka planda ne yapıyor?
- **Gerçek:** Python, bizim yazdığımız o class bloğunu okur ve onu bir "sınıf nesnesi" olarak oluşturmak için arka planda **type()** sınıfını kullanır.
- **Örnek Bilgi:** type() aslında bir fonksiyon değil, bir sınıfın kendisidir!
  - x = 5 -> x bir int nesnesidir.
  - class Person -> Person bir type nesnesidir.

BTK AKADEMİ

Meta Class Kavramı

- Neden MetaClass'a İhtiyacımız Var?
- **Problem:** Sınıf oluşturulduktan sonra (çalışma zamanında) bir şeyler değiştirmek zordur. Ama sınıf oluşturulurken (derleme zamanında) müdahale etmek istersek?
- **Örnek:** "Tüm sınıflarımın metod isimleri büyük harfle başlamalı" gibi bir kural koymak istiyorsan, bunu her sınıfta manuel yazmak yerine, sınıfı "yaratan" mekanizmaya (metaclass) bu kuralı ekleyebilirsin.

Meta Class Kavramı

- MetaClass Olmadan (Geleneksel Yöntem)
- Eğer metaclass kullanmazsanız, her sınıf için kuralları manuel kontrol etmeniz gerekir.

```
# Her sınıfta aynı kontrolü yapmak zorunda kalırsınız
class Student:
    def __init__(self, name):
        self.name = name

# Eğer kural: "İsimler büyük harf olmalı" ise,
# her class'ta bunu ayrıca kontrol etmeniz gerekir. (Çok zahmetli!)
```

BTK AKADEMİ

Meta Class Kavramı

- MetaClass İle (Otomatik Kontrol)
- Metaclass, sınıf oluşturulduğu an (daha class instance'ı bile yokken) devreye girer.

```
# 'type' sınıfından miras alıyoruz (bu artık bir fabrikadır)
class MyMeta(type):
    # 'cls' (class) oluşturulurken bu metot çalışır
    def __new__(cls, name, bases, attrs):
        print(f"Sınıf oluşturuluyor: {name}")
        # Örneğin, sınıfın tüm metotlarını kontrol et, hata varsa durdur!
        return super().__new__(cls, name, bases, attrs)
```

BTK AKADEMİ

Meta Class Kavramı

- MetaClass İle (Otomatik Kontrol)
- Metaclass, sınıf oluşturulduğu an (daha class instance'ı bile yokken) devreye girer.

```
# 'metaclass' parametresi ile fabrikayı bağlıyoruz
class Student(metaclass=MyMeta):
    pass

# Çıktı: Sınıf oluşturuluyor: Student
# (Biz hiç 'Student()' diye nesne üretmedik bile!)
```

BTK AKADEMİ